IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 26 August 2026, accepted 14 September 2026, date of publication 17 September 2026,
date of current version 23 September 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3734596

RESEARCH ARTICLE

A Combinator Library for Neural Network Implementation

RICHARD SENINGTON
School of Engineering Science, University of Skövde, 541 28 Skövde, Sweden
e-mail: richard.james.senington@his.se

This work was supported in part by The Knowledge Foundation (KK-Stiftelsen) through the Virtual Factory with Knowledge-Driven Optimization (VF-KDO) Research Project under Grant 20180011.

ABSTRACT The field of Neural Networks has evolved quickly, and there is a dizzying variety of algorithm variants. The concepts presented in publications, by comparison, are often diagrammatic, capturing the intention, which leads to a conceptual gap between the implementation and the explanation. This gap in conception and implementation also adds to issues in reproducing the results reported in many papers, due to difficulties in reimplementing the networks. In this paper, we transfer the concept of a combinator from Functional Programming and apply it to architectural concepts in neural network design. We show that many design concepts in neural networks are variations on common patterns and how they can be implemented using common underlying combinators within the PyTorch framework. We discuss the types of these combinators and how understanding these patterns can drive improved reliability in neural network implementation. Finally, we test our combinator-based approach by reimplementing ResNet34 and the Gated Recurrent Unit.

INDEX TERMS Artificial intelligence, combinators, reusability, PyTorch.

I. INTRODUCTION

Deep learning algorithms have achieved amazing results in recent years, in a wide range of applications including image processing, time series analysis, anomaly detection, and natural language processing [1].

This technology has been driven by a combination of highly parallel GPU chips and a plethora of different libraries [2], [3], [4], [5], in a range of languages to support researchers and developers. A primary concern for these tools has always been performance due to the length of the training cycles and the scale of the datasets required. The focus of researchers in the AI field has mostly been on achieving better results, learning more, learning faster, and having resulting networks that make fewer mistakes.

The success and relative ease of access to these tools and frameworks has resulted in an expansion of research into networks and, hence, the publication of a wide variety of network designs. For example, Resnet [6] made heavy use of *residual* connections and showed that using them, it is possible to build much larger and deeper networks for image classification. Another example is U-Net [7], which used *skip* connections to improve reconstruction in autoencoders. A slightly different direction of research has been *modular neural networks* [8], [9], which use modularity as a design concept to improve training, structure, or to improve reuse of trained components.

Consider the structure in figure 1. This structure shows a network for image encoding and reconstruction, using *skip* connections between paired layers on each side of the bottleneck. However, code for this usually involves the manual linking of layers, or calculation of layers to link using loops and modulo, as seen in [11] and [12]. The difference between the diagram and the code creates a conceptual gap between the design and the implementation of networks when using the standard frameworks. An additional, but related, issue is that of reproducibility. Specifically, the interest in the field has led to a rise in the number of publications. However, it is not always easy to reproduce the networks and training processes proposed in those publications, as shown in [13] and [14].

The proposal in this paper is to use combinators, a concept from functional programming, to better capture common architectural structures in neural networks, and hence close

The associate editor coordinating the review of this manuscript and approving it for publication was Stefano Caputo.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

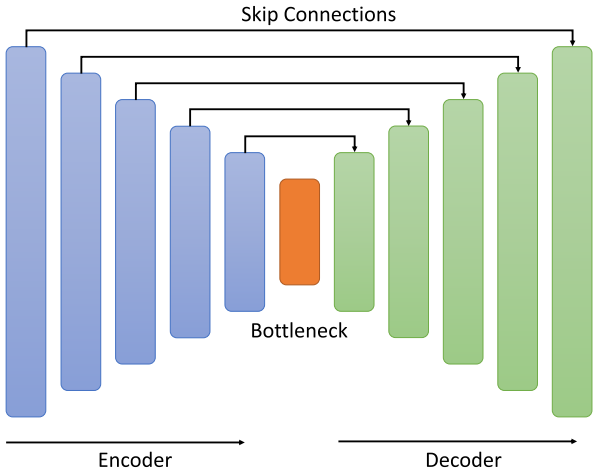


FIGURE 1. A diagram of a UNet autoencoder, using a similar approach to that seen in [10].

the implementation/conception gap. The use of combinators also allows us to highlight similarities between structures by showing that some combinators can be implemented in terms of others through appropriate parameterization. We propose that this will provide a concise notation, with high levels of reuse, that can effectively match a network's conceptual idea to its code, and we will illustrate this using ResNet as an example [6].

Combinators are a form of *higher order function* seen in functional programming, and have been employed successfully to create highly modular toolboxes [15], [16], [17]. Higher-order functions are functions that either take other functions as parameters, give back functions as results, or both. Combinators are higher-order functions that take functions, along with other data, as parameters and return functions as outputs. Hence, they can be thought of as functions that recombine code. We can think of neural networks as a special form of function, as has been pointed out by others, for example [18], [19] and also in influential blog posts [20]. The contention here is that after training, a network is a functional approximation of an underlying function that maps an input type, for example, a picture, onto an output type, for example, the classification of that picture. During training, a network can be thought of as a function that takes a parameter data structure and returns a possible approximation function. The task in training is to find a set of parameters that minimizes the error in the approximation function we are developing. Hence, the training task is also a higher-order function [18].

Since neural networks can be thought of as higher-order functions, we can recombine them using combinators. These combinators can then be used to encode common structures seen in neural network design. This approach shares some kinship with Neural Module Networks (NMN) [8], [9], which build networks from composable blocks. The difference is that NMN research looks at which modules, intended for specific tasks, should be combined, and why, while combinators look at how to combine components and recurring patterns of combination. In strongly typed functional languages, combinators can also improve safety and compiler validation through imposing strict type patterns on the components that can be recombined. While this will not be a core focus of this paper, it points to how the usage of combinators has future potential in supporting the safe implementation of neural networks. A similar direction of research has been a category-theoretic approach [21], which more directly applies to the field's mathematical underpinnings.

The usage of combinators has application in expressing neural network structures, showing similarity between structures, and also in enabling rearrangement and experimentation on these structures. Such experimentation, not using combinators, can be seen in work such as [22], where they integrated insights from transformer design into ResNet to yield a model with performance characteristics of both prior approaches.

The JAX library [4] is more functional than libraries like PyTorch, and Penzai [23] is a library that includes combinators for neural networks. This paper expands on the Penzai library combinators by providing a PyTorch library and showing the relationships between families of combinators and how they can be created by specializing common underlying functions. Researchers at Google investigated the use of combinators in tools related to neural networks, using them to structure computations for dynamic graphs needed for processing networks [24], but not the networks themselves. Combinators have also been used in relation to PyTorch previously, but in conjunction with *probtorch*, for probabilistic programming models [25].

Purely functional implementations of neural network libraries have been created, for example, Grenade [5]. An alternative approach can be seen in TypedFlow [26], which provides a typed wrapper for TensorFlow in Haskell. TypedFlow turns the operations into higher-order functions and provides functions for slicing tensors, flattening, and transposition, but does not provide a substantial library of combinators for structuring more complex networks. However, PyTorch and TensorFlow are far more commonly used in the machine learning community for application development, and of those, PyTorch is favored for research. This paper does not focus on a pure functional approach, but rather looks to demonstrate the advantages of a combinator-based approach and how combinators can be used successfully in conjunction with PyTorch.

PyTorch uses *modules* as the fundamental building block of neural networks. These modules are actually classes, and this is how PyTorch exposes and manages a network's trainable parameters. However, modules can also be called in the same way as functions; once trained, they act as functions that approximate a mathematical solution to the problem they were designed for. Hence, in this paper, modules are treated as approximations of higher-order functions, and the combinators we propose are either modules themselves that

take other modules as parameters or functions that provide further specialization. This focus on direct implementation of combinators in PyTorch also supports making these tools directly applicable to practical application development.

In order to demonstrate the concept of combinators for neural network construction, we first look to break down a standard neural network dense layer and examine what operations should be considered indivisible, and then look at what combinator is needed to rebuild the dense layer and how the two versions compare. We then look at a subset of well-known neural network types, specifically convolutional networks for vision, including variations such as residual networks and autoencoders, recurrent networks, and gated recurrent networks. Using these, we propose combinators that capture their key architectural concepts. Finally, we demonstrate the use of the combinators to rebuild the well-known ResNet network and the GRU layer type. The prototype library can be found at https://github.com/richard-senington/pytorch-combinators.

II. ATOMIC OPERATIONS

At a mathematical level, practitioners would agree that the core operations of Neural Networks include operations like vector-matrix multiplication and vectorized operations, such as activation functions, where a simple function like Sigmoid is applied pointwise to each element of the input. These operations are also supported at the hardware level via GPU technology, which leverages the inherent parallelism of the operations.

However, most frameworks discuss *layers*. This concept draws on both the original biological inspiration and the developments of the perceptron and the multi-layer perceptron. In PyTorch and TensorFlow, these are usually equated to modules, with the libraries providing a variety of prebuilt types. Generally, these library-provided components are internally optimized to improve performance, and hence, it normally makes sense to use them. We begin with this example from [27], which presents a standard layer as a PyTorch Module.

```
class MyLinear(nn.Module):
  def __init__(self, in_units, units):
    super().__init__()
    self.weight = nn.Parameter(
          torch.randn(in_units, units))
    self.bias = nn.Parameter(torch.randn(units,))

  def forward(self, X):
    linear = torch.matmul(X, self.weight.data)
                + self.bias.data
    return F.relu(linear)
```

The *__call__* method shows the process that is taking place, expressed in a conventional equational style. The process can also be thought of as a sequence of 3 operations: Matrix Multiplication, Vector addition & ReLU activation. An approach that thinks in terms of recombining these most basic building blocks can approach this in the following way;

- Create an Infix annotation, to allow for infix operations. There are several methods in python that can work, but the approach I will demonstrate is to create an annotation based on a class, that overloads the *__or__* and *__ror__* methods to capture left to right reading of sequenced operations.

```
class Infix:
  def __init__(self, func):
    self.func = func
  def __or__(self, other):
  return self.func(other)
def __ror__(self, other):
  return Infix(lambda x: self.func(other, x))
```

- Create a *seq* function that can sequence two existing neural network structures. Note that a most basic version of this could look like this;

```
@Infix
def seq(a,b):
  return Sequential(a,b)
```

  This implementation is deliberately simple to support the combinators seen in this paper, and is defined in terms of the Sequential module from the PyTorch library. However, it would be more useful (and more complex) for this function to inspect the types of incoming parameters and either recombine them or build an appropriate structure to capture their properties efficiently. For example, if both are sequences themselves, returning a single sequence containing all layers from both original networks would be preferable.
- Define some small network units to capture the atomic operations, specifically, MatrixMul, VectorAdd, and ReLU. ReLU (as shown in the example above) is special in two ways. First, it already exists, and second, it has no learnable parameters. It would be possible to create the matrix and vector layers by wrapping standard library operations, but PyTorch provides them combined as the Linear Layer. The separation presented here is for clarity of the sequence of operations.

Using the above building blocks, we can define the dense layer like so;

```
def MyDense(in_units, units):
  return MatrixMul(in_units, units) |seq|\
         VectorAdd(units) |seq|\
         ReLU()
```

This function builds the dense-layer's structure while exposing all the components in a very minimal form. The function *seq* is a first, smallest, combinator; it takes 2 existing networks and combines them into a new network.

III. COMBINATORS

We have seen the first, most basic combinator, *seq*. In this section, we look at capturing other common patterns seen in neural networks as combinators, and we highlight relationships & similarities between the patterns themselves. In section III-A we show the *autoencoder* pattern, that it is based on *seq* and we discuss that capturing it as a combinator can enable checks on the pattern of the parameters passed to it. In section III-B we look at the pattern *residual*, and then in section III-C we look at

skip and show that *residual* can be expressed in terms of *skip*. In section III-D we look at gates, and show that the same pattern needed for gates, which we call *probe*, can also be used for *skip* and *residual*. In section III-E & section III-F we look at how further decompose *probe* and how related patterns can also be seen in architectures such as *mix of experts*. In section III-H we discuss *recurrent* networks, give a combinator to make a network into a *recurrent* network and show how combining this with *probe* gives the basic pattern of a Gated Recurrent Unit (GRU). Finally, in section III-I we discuss how combinators can structure convolutional operations, and expose their internal patterns. In passing, we introduce other helper combinators such as *repeat* & *duplicate* that have general application in structuring networks.

Usually, when implementing neural network modules in PyTorch, developers pass as many parameters as needed to the network when calling the forward operation. To simplify composition, we use a single parameter that can be a tuple, list, tensor, or a combination of the above.

A. AUTOENCODERS

The first pattern we look at is the Autoencoder, due to its relative simplicity. Autoencoders are commonly used in anomaly detection, and the concept is that the input is processed by one network into a narrower internal representation, called a bottleneck. The narrow representation of the bottleneck is then used as the source to reconstruct an output from the internal representation. In anomaly detection, these networks are often trained exclusively on *good* data, such as images, and their outputs are intended to be as close as possible to their inputs. When used outside of training, the network is expected to reconstruct an example of normal data presented as input with low error. Presenting an example of an unexpected input, however, is expected to either go very wrong or produce an approximation of the normal pattern. In both cases, the network's output when processing an anomalous input fails to match the input, and anomalies are detected through this comparison.

This structural pattern, however, is just a variation on a sequential network. We can make this very explicit like so;

```
def auto_encoder(encode,bottleneck,decode):
    return encode |seq| bottleneck |seq| decode
```

Because the parameters encode, bottleneck, and decode are networks themselves, they can be of arbitrary complexity internally, so long as their inputs and outputs match correctly. When processing images, these networks are often multilayer convolutional networks in their own right. One common thread, however, is the intention that the bottleneck acts as a bottleneck, and so it should, in some way, be smaller than the encoder and decoder. A more sophisticated version of this function might include checks for this property, but we leave that detail for more complete implementations.

B. RESIDUAL CONNECTIONS

The next pattern we look at is the residual connection, which was developed to help reduce the impact of very deep networks for image recognition. This resulted in residual networks [6], which have proven to be very successful in creating scalable and effective image classification tools.

The concept behind a residual is to create a shortcut in the network: the outputs from a layer are both transformed by the immediately following layer and copied forward to a later layer, where they are added to the tensor at that point. This shortcut enables errors to propagate further back in the network without succumbing to the vanishing gradients problem.

From a combinator perspective, we create a combinator that takes a single network as its parameter, and we call this *residual*1 and implement it as a torch module.[1] This residual module can be seen as a combinator that creates a new neural network which applies its wrapped network to an input, and combines the original input with the resulting tensor.

```
class residual1(Module):
    def __init__(self, f):
        super().__init__()
        self.f=f

    def forward(self, x):
        return self.f(x)+x
```

C. SKIP CONNECTIONS

Skip connections are, as the name suggests, also a shortcut in a network between layers. They are similar to residuals but more associated with auto-encoders and U-Net [7] (a kind of autoencoder). In the U-Net example here, the approach was to use element-wise addition of the tensors, as we saw in the residual combinator. An alternative was presented in [28], where tensors are concatenated, preserving more information at the cost of increasing the number of trainable weights.

The presence of multiple variations in how the skip connection can work lends itself to the combinator strategy. Like the residual, we need a combinator that wraps a primary network. To improve generality, we then introduce a *recombination* parameter. The recombination parameter is a neural network (or a function) that combines the input tensor with the primary wrapped network's output. Our first implementation looks like this;

```
class skip1(Module):
  def __init__(self,r,f):
    super().__init__()
    self.f=f
    self.r=r

  def forward(self,x):
    return self.r(self.f(x),x)
```

[1] Standard Python style would have *residual*1 named as *Residual*1, with a capital R. However, the implementation of the combinators mix modules and functions in the definitions. For consistency, we have chosen to stay with a single style, and that is lower case, to suggest that everything is a higher-order function.

Using *skip*1, we can define two specializations. We can now re-implement residual as a specialization of skip, and call it *residual*2. We also define a version of skip using concatenation as the recombination operation, and call it *skipC*.

```
def residual2(f):
  return skip1(lambda x:x[0]+x[1],f)

def skipC(f):
  return skip1(lambda x:torch.cat(x, 0),f)
```

We have now seen the first clear relationship between two methods, that skip and residual are related in terms of their recombination function. However, we are using standard Python lambda functions to parameterize these combinators, where, for consistency, they should be acting on neural networks. To correct this, we must lift our static function into a module. At the time of writing, there seems to be a discussion online about having such a feature in PyTorch, but it has not been formally resolved, so we will create our own here;

```
class lift(Module):
  def __init__(self,f,innerNetworks=[]):
    super().__init__()
    self.f=f
    self.inner=ModuleList(innerNetworks)

  def forward(self,x):
    return self.f(x)
```

This *lift* module includes a parameter *innerNetworks*, so that if the primary function wants to include a learning component while being a natural Python function itself, the learning components can be passed in separately and exposed to PyTorch for backpropagation. Both residual and skip can be redefined to use lift for correctness, but for brevity we will not present this here.

D. GATES

Gates act as a more flexible version of the highway concept seen in residuals. They were used to create deeper networks for vision, in the same way as residuals [29], but are now widely applied, including in recurrent networks such as GRUs and in Transformers. The difference is that a residual always passes the input round the primary transformation and then does an element-wise addition, where a gateway learns a weighting to be used to sum together parts of an input;

$$output = U.path1 + (1 - U).path2$$

where U is the learned weight, a Tensor of values between 0 and 1, and paths 1 and 2 are tensors to be recombined.

The underlying pattern that we wish to capture here is the application of two neural networks to an input, to give rise to *path*1 and *path*2, and then recombine them, where the standard gateway implementation learns how to generate U and use that for the weighted summation. Hence, all three tensors $U, path1, path2$ are all computed from the same initial input. To capture this pattern, we create the module *probe*, because it will probe possibilities and then recombine the results.

```
class probe(Module):
  def __init__(self,r,f,g):
    super().__init__()
    self.f=f
    self.g=g
    self.r=r

  def forward(self,x):
    return self.r((x,(self.f(x),self.g(x))))
```

In *probe*, the recombination operations (r) is a network that takes as input the original input to the network module, and the result of applying the two other control networks g and f.

First, we look at recreating the standard gateway. In many examples, this will use a simple linear network with a sigmoid activation function, giving values between 0 and 1, allowing it to compute the U tensor. We provide this functionality by implementing a module called *linear_recombine*, making use of the |*seq*| combinator to define internal functions.

```
class linear_recombine(Module):
def __init__(self,i_shape,o_shape):
  super().__init__()
  self.r=Linear(i_shape,o_shape) |seq| Sigmoid()

def forward(self,x):
  U=self.r(x[0])
  return (x[1][1] * U) + (x[1][1]*(1-U))
```

In [29], the approach taken was to apply this linear recombination to the original tensor, and a transformation of the tensor. However, we might also transform the input twice, as in our probe, and then recombine the two. We can implement each of these using parameterization, using *Identity* to pass the input through unchanged where necessary.

```
def residual_gate(i_shape,o_shape,f):
  return probe(linear_recombine(i_shape,o_shape),
               f,Identity())

def basic_gate1(i_shape,o_shape,f,g):
  return probe(linear_recombine(i_shape,o_shape),
               f,g)
```

Now we can also redefine *skip* in terms of *probe*. The *skip* pattern does not usually apply a learned recombination; instead, it combines the parameters directly. We provide this by lifting the recombination operation and ignoring one of the parameters.

```
def skip2(r,f):
  r2=lift(lambda x:r(x[1][0],x[1][1]),
          innerNetworks=[r])
  return probe(r2,f,Identity())
```

Residual and *skipC* can both be defined in terms of *skip*2 rather than *skip*1, without any other changes.

E. MODULARIZING RECOMBINATION

In the previous section, we built the linear recombination unit as a network instance directly. This implementation was quite clear and is likely how specializations of the *probe* combinator would be used in practice. However, we can provide combinators to further break down these operations.

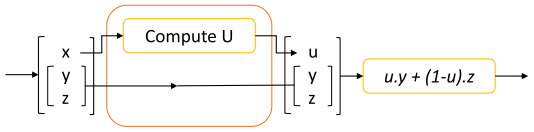


FIGURE 2. The internal pattern of the standard gateway recombination.

The general pattern we see in *linear_recombination* could be expressed as seen in figure 2.

This pattern is the same as patterns discussed in work on Arrows in functional programming [17]. The first half, where a function is applied to one element of two and the other passes through unchanged, they call this *first*. We can implement *first* for neural networks in PyTorch as follows;

```
class first(Module):
  def __init__(self,f):
    super().__init__()
    self.f=f

  def forward(self,x):
    return (self.f(x[0]),x[1])
```

This applies a learned transformation f to the first part of the input, while letting the second half pass through unchanged.

Using *first* & *lift*, we can provide an alternative version of *basic_gate* that does not require a specialized module to capture the recombination logic.

```
def basic_gate2(i_shape,o_shape,f,g):
  r = first(Linear(i_shape,o_shape) |seq| Sigmoid)
      |seq|
      lift(lambda x:x[0]*x[1][0]+
                    (1-x[0])*x[1][1])

  return probe(r,f,g)
```

F. THE FAN AND RECOMBINATION PATTERN

The *probe* combinator supports a wide range of behaviors that appear in different kinds of networks, but it is limited to only two transformations and the recombination of the results. In the case of residuals and gateways, this was sufficient. In general, however, we can imagine a situation where we might want to apply more networks to a single input and recombine all the outputs.

There are also two parts to the combinator: the application of the test networks and the recombination. Recombination has already been used in *probe* & *skip*, but it could be applied in sequence if candidates for recombination are first computed. This is the concept of *fan*, which will fan out options. To support later recombination, *fan* must also return the input as one of its outputs. The return is a tuple containing the input tensor and the possible candidates.

There are two variations to consider, the situation where the transformation for the candidates all give back tensors of the same shape, and the situation where the tensors are of different shapes. When all tensors are the same, the candidates can be stored as a list. If the candidate types are different, they should be a tuple. Hence we propose two variations; *fan_l* and *fan_t*. For implementation, we take advantage of Python's weak typing to apply a function in a list comprehension and then convert the result to a tuple.

```
class fan_l(Module):
  def __init__(self,fs):
    super().__init__()
    self.fs=ModuleList(fs)
  def forward(self,x):
    return (x,[f(x) for f in self.fs])

class fan_t(Module):
  def __init__(self,*args):
    super().__init__()
    self.fs=ModuleList(args)
  def forward(self,x):
    return (x,tuple([f(x) for f in self.fs]))
```

Using these would reduce *probe* to a sequencing of a fan variation with the recombination operation.

These two have different specific uses in existing well-known architectures. As seen in the previous version of *probe*, fan_t would be used when we have a small number of transformations that might not return tensors of the same shape. For example, in *residual* and *skip*, we typically have only two transformations, and one is *Identity*. So fan_t is needed for the combinators we have already considered. It should be noted that in most cases, when using *skip*, the shape of the tensors to be recombined is the same, though there are only two of them. However, the approach taken using these combinators means that this need not be the case, so long as the recombination network understands how to process both inputs.

By comparison, fan_l suggests a larger number of operations, all of which will definitely give back the same shape of tensor. This pattern is seen in the concept of Mix of Experts, which has been used effectively by DeepSeek V3 [30]. This is where we have several networks, each specializing in different transformations, and we select which to execute at runtime to achieve the best results. For the selected transformations, the results are then recombined. We will not address the question of how to select networks here, though it is usually a neural network operation itself.

G. REPEAT AND DUPLICATE

When creating networks, it is often necessary to apply the same structure several times, either in series or to different inputs. For example, residual networks often repeat the same residual block type in sequence. In contrast, convolutional networks apply the same-shaped kernel multiple times to the same input, with each kernel learning different patterns. To provide for these use cases, we can implement the following combinators. To apply a structure many times in sequence, we implement *repeat*, and to apply different instances of the same structure to an input many times, we implement *duplicate*. Both have the same input parameters: an integer n specifying the number of times to repeat a network, and an original version of the network structure to be repeated. To create new instances of networks, we use

the Python *deepcopy* function; however, it is important to note that this would also retain the weights of any matrices copied, which would impact the behavior of training. To correct for this, randomize or otherwise reset the network weights before training begins.

```
def repeat(n,f):
    x=copy.deepcopy(f)
    for _ in range(n-1):
      x=x |seq| copy.deepcopy(f)
    return x

class duplicate(Module):
  def __init__(self,n,f):
    super().__init__()
    self.fs=ModuleList([copy.deepcopy(f)
                        for _ in range(n)])

  def forward(self,x):
    return [f(x) for f in self.fs]
```

H. RECURRENT NETWORKS

Recurrent networks are described as those that have a recurrent connection within them, so that the output, or part of it, is fed back in as part of the input. This is considered a memory, so the network learns what to remember about a series of inputs, what will be important later, to influence a final output. There are several varieties, such as standard recurrent layers, GRUs, and Long Short-Term Memories (LSTM), which use gating mechanisms to decide what to remember and what to forget. In the frameworks, each of these is included in the standard libraries. However, it is often difficult to change the internal layer structure without implementing new versions. Combinators can help here by both exposing the structures and also enabling *parachuting* of changes to the sub-structures into the code.

The core combinator needed is one that makes a network recurrent. The combinator requires an initial memory state, but the resulting network acts only on individual inputs, and can be further combined into other larger networks. It should be noted that to perform training, it is necessary to keep track of which layers are recurrent, so that the memories can be reset, but this is an implementation detail related to training and is not addressed further here.

```
class recurrent(nn.Module):
  def __init__(self, initial_m,f):
    super().__init__()
    self.f = f
    self.memory=initial_m

  def forward(self, x):
    self.memory = self.f((x, self.memory))
    return self.memory
```

As an example of how to use this combinator, we can implement a simple RNN layer. These use learned transformations to scale and reshape the tensors so that they are the same shape. Then the outputs can be added together, element-wise, to give the unit's final output. This is the same pattern as the *probe*, previously seen. We cannot use *residual*, because we need to have two different learned transformations for the two inputs, but the recombination operation is the same. This example also includes two special operations, *get_input* and *get_memory*, which access different parts of the input tuple, and the final output has a Sigmoid activation applied.

```
def get_input():
  return lift(lambda x:x[0])

def get_memory():
  return lift(lambda x:x[1])

def simple_recurrent(i_size,m_size):
  f=get_input() |seq| Linear(i_size,m_size)
  g=get_memory() |seq| Linear(m_size,m_size)
  r=lift(lambda x:Sigmoid(x[1][0]+x[1][1]))
  return probe(r,f,g)
```

We can make this network into a recurrent unit in the following way;

```
m_size=5
i_size=3
clear_m=zeros([m_size])
rnn=recurrent(clear_m,
              simple_recurrent(i_size,m_size))
```

It is also possible to extend the simple recurrent unit implementation shown here by passing f and g as parameters. This allows the programmer to define more complex networks by modifying the parameters rather than changing the common implementation. Therefore, the combinator enables the *parachuting* of code into the framework, a feature common to functional frameworks.

To further illustrate the use of parachuting, we present a simplified implementation of a GRU, including making it a recurrent unit through the use of the *recurrent* combinator. A high-level intuition of a GRU is that it is a learning gate that integrates a current memory with a candidate memory generated from the new input and the old memory. The input, like the simple recurrent network above, will be a tuple of the new input and the memory. The simple implementation we provide requires two key network parameters, one providing the u-tensor (*u_comp*), and the other providing the memory candidate (f). Using the parameter *u_comp* we construct a recombination operation r, by first applying *u_comp* and then the linear weighted summation of the memory and candidate. The current memory is provided by a function *get_mem*. This approach allows a user to parachute their own networks for computing the weighting between the options and for creating the new memory candidate within the common structure. Finally, the parameterized probe is passed to *recurrent*, where we have fixed the history length parameter to 5 for this example.

```
def gru(u_comp,f,clear_m):
  r = first(u_comp)
      |seq| lift(lambda x:x[0]*x[1][0]+
                  (1-x[0])*x[1][1])

  return recurrent(clear_m,
                   probe(r,f,get_memory()))
```

I. CONVOLUTIONS

Convolutions, as they are usually used in neural networks, consist of a small transformation that is applied in a regular

pattern on sections of the input. They are well known in image processing where the *kernel* acts as a learning pattern recognizer, able to learn to identify useful patterns, or lack of them, in the image. The operation itself is usually described as a point-wise multiplication and addition of all values in the input segment with the values in the kernel, which is equivalent to flattening the input and applying a dot product. In images, segments are usually square and selected in a grid, moving a specific number of units (pixels) each time, called a stride. The use of the grid enables the process to maintain the relationship between positions in the input while recognizing patterns.

Two elements can be exposed to improve flexibility. The first is how the input is divided into segments. The grid is one method, and it works very well on image data, which is itself a grid. However, when the data does not form a clean grid, other sampling methods are needed, such as those seen in Graph Convolutional Network research [31]. The second part is the pattern-matching process. In the standard formulation, this will always be a form of flatten and dot product, but other methods exist here as well [32]. The core pattern is that there is a single output indicating the level of pattern recognition.

These variations suggest the following combinator, one that combines a sampling mechanic with a pattern recognizer. In normal usage, the sampling mechanism will be the grid sampler, and the recognizer will be the dot-product. The output would be flat if we stopped there, so we also add a reshaping operation.

```
class convolution(Module):
  def __init__(self,sampler,recognizer,reshape):
    super().__init__()
     self.sampler=sampler
     self.recognizer=recognizer
     self.reshape=reshape

   def forward(self,x):
     samples=self.sampler(x)
     y=[self.recognizer(s) for s in samples]
     return self.reshape(x,y)
```

A 'normal' image convolution would be implemented in the following way. We have not provided all the code for *grid_sample* or *reshape*, but both should be self explanatory. For the other functions, *duplicate* was covered previously, and *Flatten* is from the PyTorch libraries, and *dot_product* is a layer that captures the kernel matrix and the dot product operation, provided by the PyTorch libraries.

```
basic_convoltion(k_size,num_k,stride,outputshape):
  dp=Flatten() |seq| duplicate(num_k,
                    dot_productOp(k_size*k_size),)
  gs=grid_sample(k_size,stride)
  rs = reshape(outputshape)
  return convolution(gs,dp |seq| ReLU, rs)
```

IV. COMBINATOR TYPES

This paper has been about finding combinators for neural networks to make common patterns explicit. The types of these combinators have not been discussed so far, partly because this was not the paper's emphasis and partly because strict/static types are not well supported in Python, especially dependent types. However, combinators are pure functions, hence we can describe their types, and this can give some insight into the structures that they recombine, as pointed out by other authors [20]. So, at this stage, we sketch some of the types of combinators we have seen.

First, we will discuss the types of neural networks. We know that neural networks represent learning functions from an input type to an output type, where, in practical systems, both will usually be Tensors. The input and output types of a network are expressed as 2 dependent types. Neural networks must also include the learning parameters as parameters of the network type. However, since PyTorch Modules hide this aspect, we will, for simplicity, omit it as well. The types A and B will often be tensors or composed of tensors. Where tensors appear, we assume that they also represent the shape of the tensor. Hence, a 2d Tensor 10×10 should not be the same type as the 2d tensor 11×11. We have to admit to avoiding the issue of how to express these types and work with them efficiently and instead point to others who are exploring this issue more completely [33], [34] In short, we will not include the tensors or their shapes right now, but leave them as abstract dependent types. Hence, it is possible to have various patterns of network included in combinators, including;

- a network from one tensor to another; NNet A B.
- a network that returns 2 outputs; *NNet A* (B, C), where the type of the output is a tuple.
- a network that takes 2 types of input; *NNet* (A, B) C.
- a network that takes a list of possible inputs, all the same type; *NNet* $[A]$ B
- a network where the input is the same type and shape as the output; *NNet A A*

We express the types of the combinators in terms of these *NNet*s. The most basic is the binary sequence operation, which acts like function composition, but for trainable functions. As already pointed out, the autoencoder is an elaborated case of sequencing.

$$\begin{aligned} \mathit{seq} \quad &: \quad (\mathit{NNet}\ A\ B, \mathit{NNet}\ B\ C) \\ &\quad \rightarrow \mathit{NNet}\ A\ C \\ \mathit{autoencoder} \quad &(\mathit{NNet}\ A\ B, \mathit{NNet}\ B\ C, \mathit{NNet}\ C\ D) \\ &\quad \rightarrow \mathit{NNet}\ A\ D \end{aligned}$$

The type of the residual combinator is determined by the requirement to combine, through pointwise addition, the output of the wrapped network and the original input. Hence, the types of both the wrapped network and the resulting network have to be the same.

$$\begin{aligned} \mathit{residual} \quad &: \quad \mathit{NNet}\ A\ A \\ &\quad \rightarrow \mathit{NNet}\ A\ A \end{aligned}$$

It should be noted that, while this function type shows the constraint on what transformations can be skipped, other researchers have gone further and shown that only a certain subset of transformation functions can be used effectively [35]. A further index of transformations that can be transformed in this way, and the resulting properties, would be interesting for further work.

Skip connections are a generalization of the residual connection. This combinator takes 2 networks: the first is the recombination operation, which takes the original input type and the transformed result and produces the final result; the second is the network being routed around.

$$skip \quad : \quad (NNet\ (A, B)\ C, NNet\ A\ B) \rightarrow NNet\ A\ C$$

It is interesting to note that this type ok *skip* is also the type of the S combinator from the Lambda Calculus [36], [37], when lifted to act on learning functions. Given the similarities we have already shown between *skip*, *residual*, and the more general *probe*, there are further similarities to the S combinator in the later examples too.

When parameterized with the pointwise addition operation, this type would become restricted back to only operating on Tensors of the same shape.

Gates are similar to *skip* and *residual*, in that they recombine other tensors to give the final output. Unlike *skip* and *residual*, it is required that this recombination operation be learned, as the gates work by learning how to choose what to recombine. Additionally, *gate* is often used in the context of recurrent networks, meaning that only a part of the input is likely to be recombined into the output (the previous memory). To capture this in a general way, we have proposed passing two networks to the *probe* combinator, and then learning how to recombine their outputs.

$$probe \quad : \quad (Net\ (A, (B, C))\ D, Net\ A\ B, Net\ A\ C) \rightarrow Net\ A\ D$$

As with *skip* to *residual*, we can see that the correct parametrization of *gate* (the first operation should be identity) can now give rise to *skip*. Hence, *probe* also gives rise to *residual*.

In the code examples, we showed how *gate* can be built from the more general *fan* combinators. The two combinators described would have the following types.

$$fan_l \quad : \quad ([Net\ A\ B]) \rightarrow NNet\ A\ (A, [B])$$

$$fan_t \quad : \quad (Net\ A\ B, Net\ A\ C) \rightarrow NNet\ A\ (A, (B, C))$$

The type of the recurrent combinator would be more complex than the other combinators shown so far, in that it would need to include additional parameters such as history length as an integer. However, in order to focus on the key features of the combinator, we provide the following limited form of the recurrent combinator. We have used this model because it is the approach taken in the standard layer types: the recurrent layer uses its output memory directly as its output, as well as feeding it back in.

$$recurrent : \quad (M, Net\ (A, M)\ M) \rightarrow NNet\ A\ M$$

There is a similarity to arrows that is visible in this type, specifically the concept of a loop [38]. In the functional loop, the type would be;

$$loop \quad : \quad (Net\ (A, M)\ (B, M)) \rightarrow NNet\ A\ B$$

This type would make explicit that the network has an input and output and memory types, and that when we make it recurrent, we specifically bind the memory type away. The original version of loop does not include an initialization of M, where our recurrent combinator does.

Finally, we look at the type of *convolution*. For the combinator version of this, we argued that we would 'sample' the input to create the components that the kernel would be applied to. In a normal setup, this sampling would be done in a static grid, and only the kernel is learned. By using combinators, the sampling action can be abstracted into another learnable function, which might be a static grid, another static shape, or another neural network. Similarly, the process of taking the results of the kernel matching and building the output tensor can now be a form of learning function.

$$convolution \quad (Net\ A\ [B], Net\ B\ C, Net\ (A, [C])\ D) \rightarrow NNet\ A\ D$$

V. COMPARING IMPLEMENTATIONS

The purpose of this work is not to provide new kinds of neural networks, but to propose a different way of structuring them to enable more formal, structured building blocks. However, it is important to show what happens when these combinators are used. One difference from the code previously shown is present in the library used for these tests; the *seq* operation inspects parameters to allow it to combine sequential operations and reduce nesting. We show the differences in how the networks are expressed and compare their performance in terms of training times and error levels.

Three tests have been carried out and reported on. Two are be on the ResNet34 [6] algorithm for image recognition and one is an implementation of the GRU with a network using the GRU for a time series dataset. ResNet34 was first be tested on a subset of the PlantVillage dataset [39] and then on the CIFAR-100 dataset [40]. The subset that is used of the PlantVillage dataset is the healthy leaves only, giving 12 classes, and the whole of the CIFAR-100 dataset is used giving 100 classes. The GRU is tested on the UCI Human Activity Recognition (UCI-HAR) dataset [41]. The UCI-HAR dataset has 6 classes of human activity that can be recognized.

A. COMMON EXPERIMENTAL SETUP

All experiments were run on a workstation with an NVIDIA GPU running CUDA 12.5. The machine had 140GB of RAM, and the datasets were cached in memory.

Each network was implemented using the combinator library over PyTorch. The combinator implementation was then compared against a standard implementation. The standard version of ResNet34 will be drawn from the TorchVision implementation [42]. The GRU was tested by creating a network with 2 GRU layers and an output projection transformation. The standard version will use the PyTorch GRU layer, and the combinator version will use a combinator implementation of the GRU layer. Code for the

combinator versions is presented, and the number of trainable parameters is compared.

The testing process for these networks used statistical comparison over multiple training runs of each network. Each network was trained 50 times for 10 epochs with the network parameters randomized between each training run. The 50 training runs were chosen to be neither overly sensitive to outliers, as might be seen at 25 tests, nor so computationally heavy as 100 training runs. 10 epochs were chosen as a fixed standard, rather than running each until it fully converged.

Statistical testing across multiple runs is not ideal, as separating the influence of the initial weights of the network from the training process is an issue. However, we cannot guarantee that the weights are set in exactly the same order across the network implementations, so simply locking the random seed would not be enough either. Hence, we have tested with multiple runs.

For each algorithm we will compare;

- the number of trainable parameters of the combinator implementation and the standard implementation, with the expectation that they will be the same,
- the memory usage of the combinator implementation and the standard implementation where data will be provided through the *torch.cuda* memory reporting and clearing functions,
- the training times of the combinator implementation to the standard implementation, presenting the result as histograms,
- the losses during training, averaged over the 50 test examples, presented as a line chart
- the accuracy of the trained networks, presented as confusion matrices or tables where there are too many classes.

B. RESNET 34

ResNet's innovation was the use of residual connections within the network. The implementation is as a series of *blocks*, many of which could almost be called transformers, since the input and output shape are the same. Because the inputs and outputs are identical, the input can be added to the output, creating the gradient highway and enabling deeper networks. Between these groups of these residual wrapped blocks is another kind of block that changes the shape of tensors through changing the stride parameter of the convolutions. The structure of Resnet34 is presented in Figure 3, where the groups of blocks are placed side by side.

We do not reproduce the code from the standard implementation seen in the TorchVision library [42]. However, it uses several conditions, for-loops, and method calls spread across several classes, which we argue makes it harder to follow.

To create our implementation, we use standard operations such as Convolutions, batch normalization, and ReLU provided by the standard PyTorch library. We did not use our own convolution combinator because the added flexibility was unnecessary in this example, and standard layers are

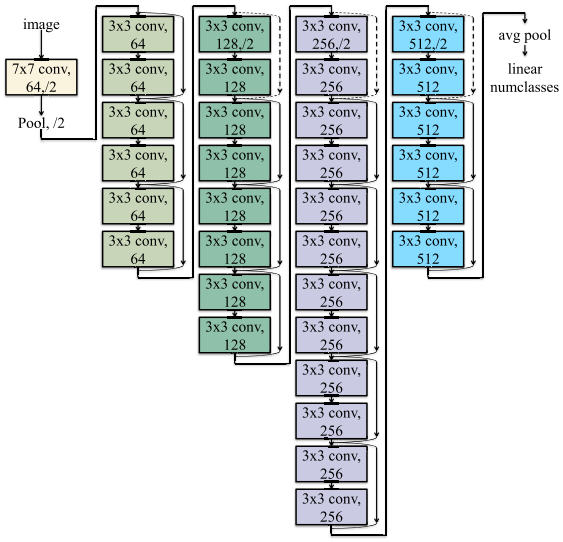


FIGURE 3. Diagram of the structure of ResNet34, loosely based on the diagrams found in [6].

easier for practitioners to understand without detracting from the structuring of the model using combinators. All parameters for these layers were determined by studying the standard implementation in TorchVision [42].

Our implementation is based on creating a set of 6 blocks, corresponding to the groups of blocks described in the original ResNet Paper [6]. To implement each block, we use *seq* to combine layers appearing in series, and the residual combinator to provide residual connections within block definitions. The result is a model that closely follows the algorithm's diagrammatic plan.

```
entry_unit = Conv2d(3, 64,
                    kernel_size=7,
                    stride=2,
                    padding=3,
                    bias=False)
             |seq| BatchNorm2d(64)
             |seq| ReLU(inplace=True)
             |seq| MaxPool2d(kernel_size=3,
                              stride=2,
                              padding=1)
unit1 = repeat(3,basic_block(64))

unit2 = reshaping_block(64,128)
        |seq| repeat(3,basic_block(128))

unit3 = reshaping_block(128,256)
        |seq| repeat(5,basic_block(256))

unit4 = reshaping_block(256,512)
        |seq| repeat(2,basic_block(512))

exit_unit = AdaptiveAvgPool2d((1, 1)) |seq|
            Flatten(1) |seq|
            Linear(512, numClasses)

return entry_unit |seq|
       unit1 |seq|
       unit2 |seq|
       unit3 |seq|
       unit4 |seq|
       exit_unit
```

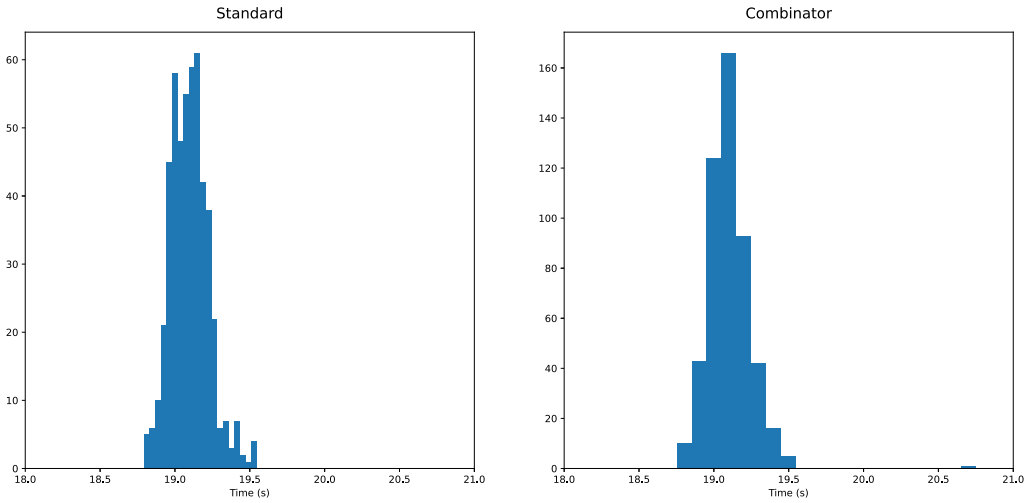


FIGURE 4. A histogram showing a comparison of the time taken to perform epochs of training for the PlantVillage dataset.

This implementation uses two specialized functions to implement the standard residual blocks, as well as residual blocks that downscale the width and height of the input tensors. These are provided using the combinators we have discussed in this paper, and some standard operations from PyTorch.

```
def basic_block(n):
  return residual(Conv2d(n, n,
                         kernel_size=3,
                         stride=1,
                         padding=1,
                         bias=False)
                  |seq| BatchNorm2d(n)
                  |seq| ReLU(inplace=True)
                  |seq| Conv2d(n, n,
                                kernel_size=3,
                                stride=1,
                                padding=1,
                                bias=False)
                  |seq| BatchNorm2d(n))
          |seq| ReLU(inplace=True)

def reshaping_block(a,b):
  f= Conv2d(a, b,
            kernel_size=3,
            stride=2,
            padding=1,
            bias=False)
     |seq| BatchNorm2d(b)
     |seq| ReLU(inplace=True)
     |seq| Conv2d(b, b,
                  kernel_size=3,
                  stride=1,
                  padding=1,
                  bias=False)
     |seq| BatchNorm2d(b)
  downsample = Conv2d(a, b,
                      kernel_size=1,
                      stride=2,
                      bias=False)
               |seq| BatchNorm2d(b)
  recombine=lift(lambda x:x[1][0]+x[1][1])
  return probe(recombine,f,downsample)
         |seq| ReLU(inplace=True)
```

This code produces a model with the same number of trainable parameters as the standard model found in TorchVision. The memory usage for the allocation of the combinator and standard networks was the same, taking into account the number of classes and hence the output layer sizes. For the PlantVillage dataset the networks required 86.175mb of memory and for CIFAR-100 86.355mb. This indicates that in this example the functions creating different networks are not adding substantial overhead to the memory usage.

1) TRAINING WITH PLANTVILLAGE

Figure 4 shows how long each model takes to train a single epoch. This shows that the training times in both models are broadly normally distributed around a similar mean, though the combinator model has an outlier at 20.74 seconds. This outlier was investigated, and it was found to be the time taken for the very first epoch of the data-gathering cycle, suggesting an unexpected setup cost. Figure 5 shows the average training loss after each epoch. The pattern between the two is very close and nearly identical at the end, suggesting that the training behavior of these two networks is very similar, as expected. The average time per epoch is 19.11s for the combinator implementation and 19.10s for the standard implementation. The training dataset was 12068 images, giving a throughput of 0.0016s per image in each case.

After training for 10 epochs, each resulting network was evaluated against a validation set included in the dataset. The confusion matrices of the standard implementation and the combinator-based implementation can be seen in Figure 6. Each confusion matrix represents the average classification capability across the 50 trained networks. Some anomalies are present, for example, where the image is of Soybean leaves, the predictions of Corn in the combinator implementation are much higher than in the standard version. However, it should be noted that the classes were unbalanced, with more soybean images than any other. Hence, we also present a version in which each error is expressed as a

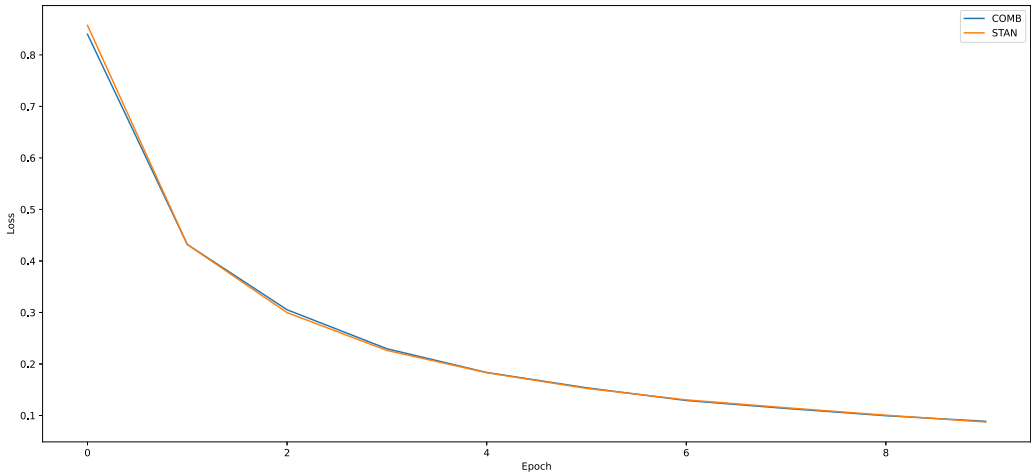


FIGURE 5. A comparison of average training process losses while training on the PlantVillage dataset.

percentage of the class size, as seen in Figure 7. The differences remain, but are less obvious. It should also be noted that, typically, when a substantial misclassification occurs, such as Potato for Soybean, both models exhibit substantial error in the same location, though the error level differs. Since the models had not fully converged at this stage and the issue of weight randomization had been previously noted, we argue that the strong similarity between these matrices suggests that the models are behaving similarly, as we intended.

2) TRAINING WITH CIFAR-100

The second test used the larger CIFAR-100 dataset, with 100 classes. Figure 8 shows a histogram comparing the training times. As with the PlantVillage test, the training times show a similar range, though both the combinator and the standard implementation are slower per epoch, reflecting the increased dataset size. The average time per epoch is very similar for both implementations, with the Combinator implementation taking 104.94s and the Standard implementation taking 104.85s. The difference is only clear when considering the full dataset, which has 50000 images in the training set, giving a throughput of 0.0021s per image in both cases. The distributions of the times per epoch are not exactly the same, with the Combinator implementation showing slightly greater deviation. This is slower than the throughput found in the smaller dataset for PlantVillage, though since it is similar for each implementation it is more likely caused by the training loop than the neural network code.

Figure 9 shows a comparison of the average training loss of the two implementations when trained on the CIFAR-100 dataset. They show nearly identical training rates, though they have not converged after the 10-epoch test cycle.

Because of the number of classes present in this dataset, it is not possible to compare the evaluation of the training processes using a confusion matrix, as was the case with PlantVillage. Instead, a comparison can be made of the per-class F1-scores to indicate whether the implementations behave similarly, as expected given the training losses. Table 1 shows the results of a subset of the classes of the dataset. The classes were ranked by average F1-score,

TABLE 1. A table showing a comparison of F1-scores (to 3 d.p.) for a selected subset of classes from the evaluation of the ResNet implementations, tested on the CIFAR-100 dataset. The selection includes the bottom 7, top 7, and middle 8 classes, based on the rank after sorting by the combinator implementation's average F1-score.

Class Name	Combinator		Standard	
	Rank	F1-score	Rank	F1-score
otter	1	0.221	2	0.229
seal	2	0.234	1	0.218
lizard	3	0.281	3	0.284
bear	4	0.283	4	0.299
girl	5	0.301	7	0.323
squirrel	6	0.314	6	0.315
woman	7	0.317	8	0.325
forest	47	0.525	50	0.546
pine_tree	48	0.530	46	0.530
dinosaur	49	0.531	47	0.531
mushroom	50	0.531	48	0.534
bed	51	0.537	54	0.560
house	52	0.540	49	0.539
spider	53	0.546	55	0.565
fox	54	0.547	52	0.550
orange	94	0.771	94	0.762
chair	95	0.779	95	0.781
motorcycle	96	0.787	96	0.791
plain	97	0.794	97	0.795
sunflower	98	0.800	98	0.795
wardrobe	99	0.811	99	0.826
road	100	0.851	100	0.854

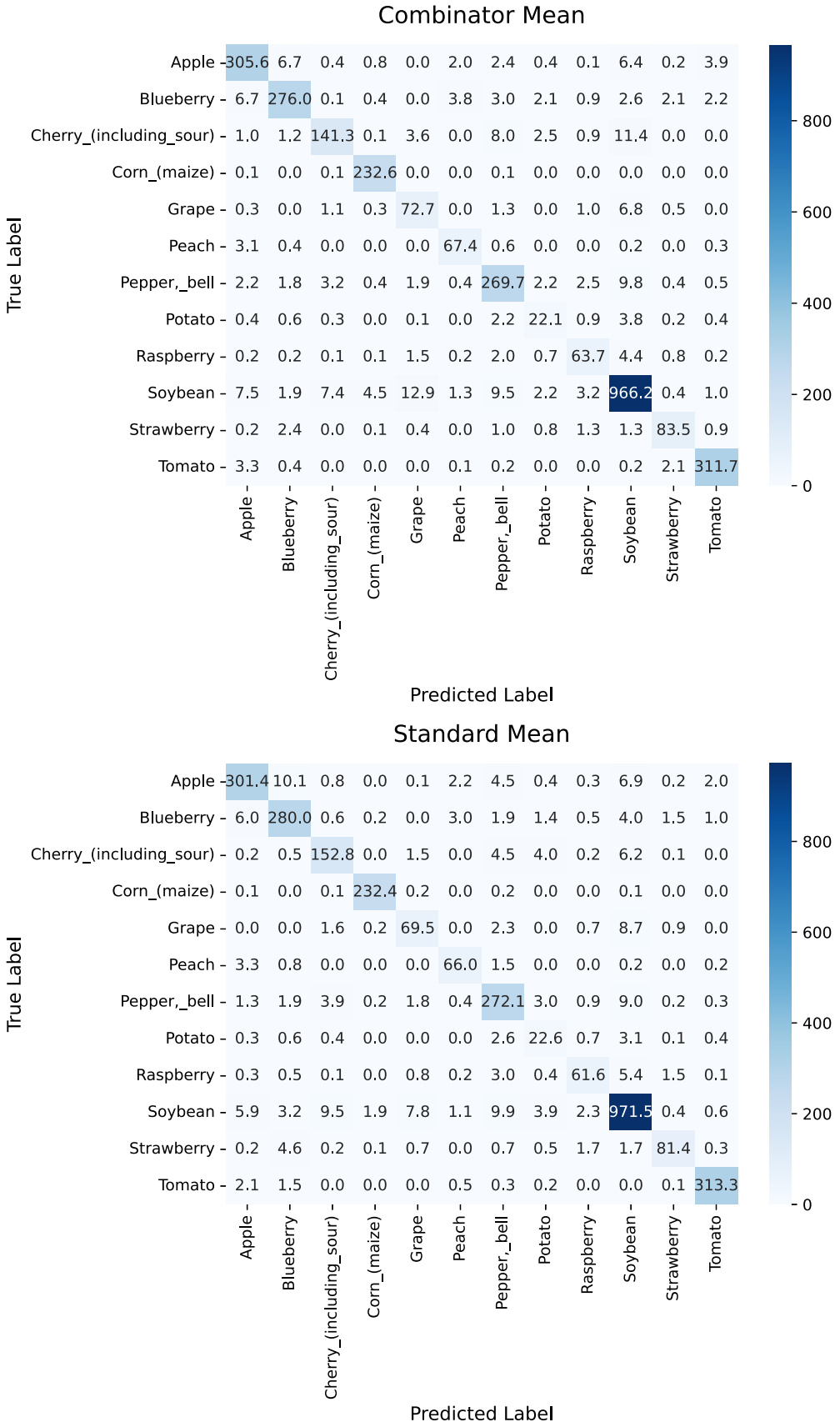


FIGURE 6. A comparison of the average confusion matrix of the final trained networks, using the PlantVillage evaluation dataset.

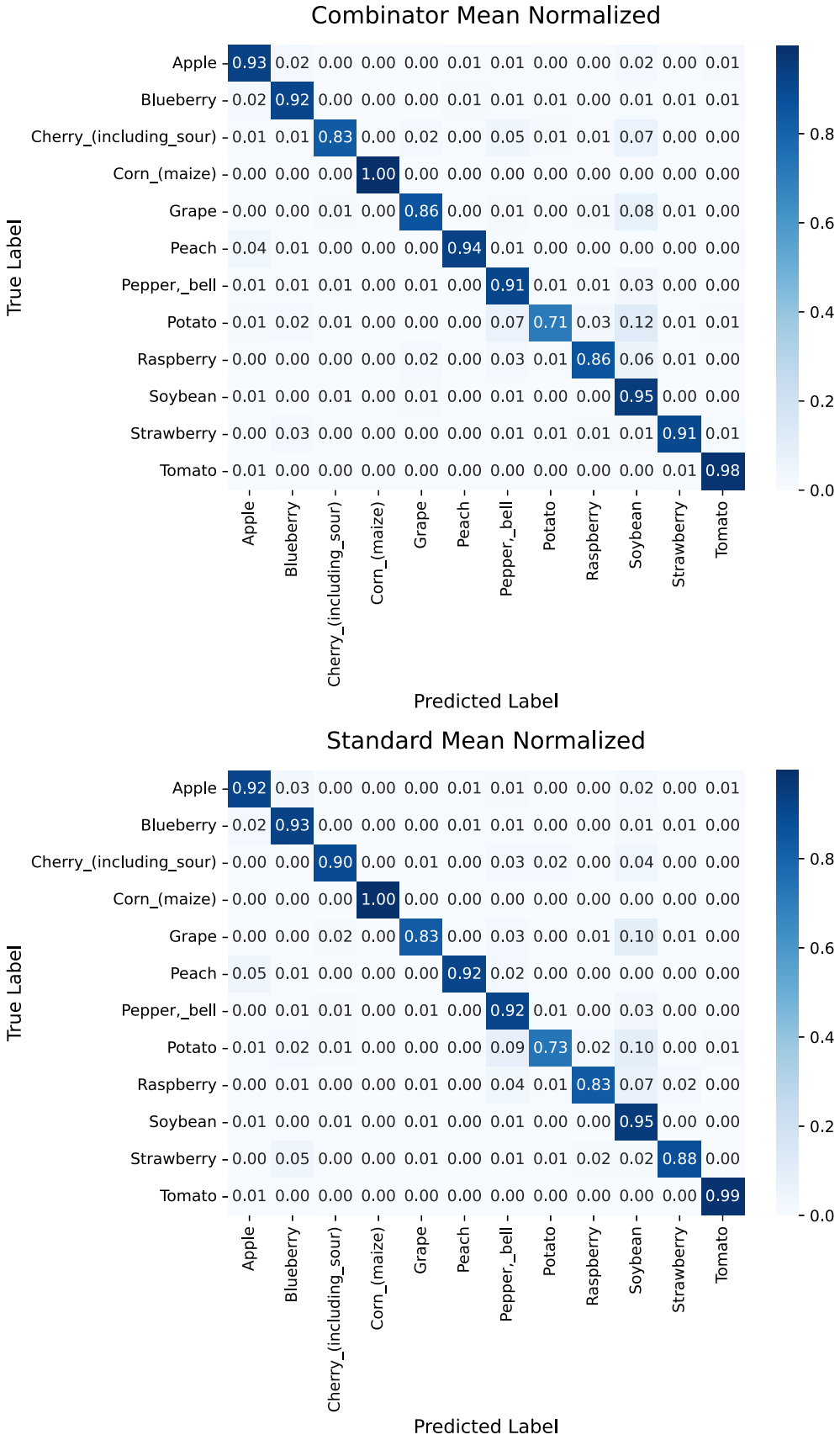


FIGURE 7. A comparison of the average confusion matrix, normalized by class size, of the final trained networks, using the PlantVillage evaluation dataset.

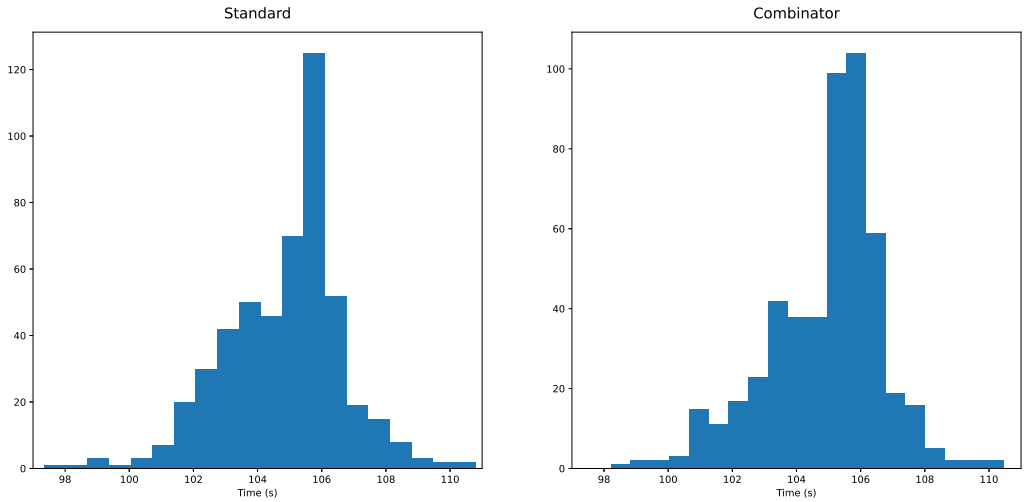


FIGURE 8. A histogram showing a comparison of the time taken to perform epochs of training for the CIFAR-100 dataset.

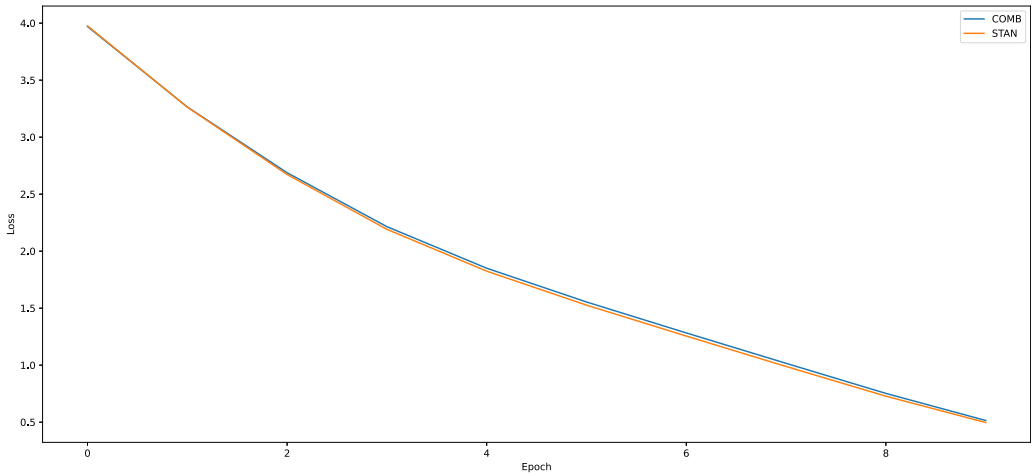


FIGURE 9. A comparison of average training process losses while training on the CIFAR-100 dataset.

where 1 has the worst score and 100 the best. The subset presented was created by taking the 7 worst, the 7 best, and the middle 8, based on the ranks drawn from the combinator implementation. The class ranks from the standard ResNet implementation are also shown for comparison. What is seen is that there are differences in F1-scores between the implementations and differences in rank order. However, the ranks are typically very similar, reflecting high similarity in F1-scores and, hence, in implementation behavior.

C. GATED RECURRENT UNIT

The network design used for this test is a small system with 2 GRU layers and an output projection from the hidden size down to the number of output classes. The input size of the network was 9; the number of classes was 6 (reflecting the Human Activity Recognition dataset); and the hidden size was set to 64. These values were set as default values for parameters in the implementation.

The network version considered standard for this test was implemented as a PyTorch module, with an appropriately parameterized GRU and a linear layer for the output transformation. The *forward* method handled the network logic.

The basic pattern of the GRU is a recurrent gateway, that can learn how to recombine the existing memory with a candidate update for the memory. This was captured in a combinator called *gru* which takes 2 networks as the primary parameters, the learnable computation for the recombination weights, and the learnable computation of the update candidate. The combinator also takes a static function to give a *blank* memory. Of course, each of these functions can be specified in a range of ways; for example, the memory could be initialized as zeros or randomized. The *gru* combinator ends by combining the inputs using the recurrent combinator.

```
def gru(u_comp,f,clear_m):
  r = first(u_comp) |seq|
    lift(lambda x:x[0]*x[1][0]+
                  (1-x[0])*x[1][1])
return recurrent(clear_m,
                 probe(r,f,get_memory()))
```

The basic *gru* combinator captures the common pattern of the GRU but omits the details shown in standard implementation descriptions. These details are provided by parameterizing the *gru* function. The standard design first applies a forget transformation to the memory before combining the result with the input. The forget process and the combination with the input is provided by the *forget* function, which only takes the input and hidden layer sizes. The *forget* and *gru* combinators are then used by *c_GRU* (for combinator GRU) to implement a GRU following the standard logic. The *clear* operation provides a way to initialize a blank memory. This version of *clear* includes a hard-coded batch size (64) for simplicity in this example.

```
def forget(i_size, h_size):
  c_r = first(Linear(i_size,h_size)) |seq|
        second(Linear(h_size,h_size)) |seq|
        lift(lambda x:x[0]+x[1]) |seq|
        Sigmoid()

  r= first(c_r) |seq|
     lift(lambda x:(x[0]*x[1][0],x[1][1] ))

  return probe(r,
               get_memory() |seq|
                 Linear(h_size,h_size),
               get_input() |seq|
                 Linear(i_size,h_size) ) |seq|
          lift(lambda x:x[0]+x[1]) |seq|
          Tanh()
def c_GRU(i_size, h_size):
    clear = lambda: torch.randn(64,h_size)
    e = first(nn.Linear(i_size,h_size)) |seq|
        second(Linear(h_size,h_size)) |seq|
        lift(lambda x:x[0]+x[1]) |seq|
        Sigmoid()

    f=forget(i_size,h_size)

    return gru(e,f,clear)
```

The test network, consisting of 2 GRU layers and an output transformation, is implemented in the function *finalNetwork*, which is a problem-specific model rather than a general-purpose combinator. Hidden GRU layers have the same input, output, and memory shapes, and can therefore be quickly chained together with *repeat*. The first layer, however, must be created specially, with the input size specified by knowledge of the dataset. The network ends with a standard linear layer that transforms the GRU output into a vector matching the number of classes in the problem.

```
def finalNetwork(i_size=9, h_size=64,
                 num_layers=2, num_classes=6):
    x= c_GRU(i_size,h_size) |seq|
       repeat(
         num_layers-1,
         c_GRU(h_size,h_size))
       |seq|
       Linear(h_size,num_classes)
    return x
```

Two implementation issues arose while developing this larger-scale test that were not addressed while describing the basic concept of the recurrent combinators earlier. The first is the ability to reset the memory of a contained recurrent unit, so that we can reset during training. The approach used was to add each recurrent operation to a reset manager class that understood how to reapply the memory initialization function. A manager class was then told about all recurrent units in a given network and could reset one or many at any time.

The second issue is that of managing batches and serializing data when training the recurrent network on standard datasets. The standard implementation expects a Tensor consisting of a batch of time series, and it processes these internally. The combinators lack this at present, so during training the dataset had to be processed and fed into the network. Because this was a series of Python operations, it was quite slow.

The standard and combinator implementations had the same number of trainable weights when given the same parameters. The memory usage for the allocation of the combinator network was 0.163mb after the load was completed and the standard implementation required 0.160mb. This indicates that the GRU used slightly more memory, which is not unexpected given the extensive use of the wrapping combinators such as *probe*.

1) TRAINING WITH UCI-HAR

Figure 10 shows the training times per epoch for the two implementations. The average time per epoch for the combinator implementation was 21.1s, while the average for the standard implementation was 0.42s per epoch.

While the training times show a clear limitation of the current combinator library, the training losses are much closer. Figure 11 shows that both implementations perform similarly, though the combinator implementation takes slightly longer to converge and stabilizes at a slightly higher loss.

Finally, Figure 12 shows the averaged confusion matrices for the Combinator and Standard implementations. These matrices show similar patterns, including the classes that the networks get wrong. Overall the combinator networks appear slightly less correct, and this is in line with the training losses seen in Figure 11, which suggested that the combinator implementation was converging at a weaker optimal. Despite these differences, both networks behaved in a very similar way in this test.

D. DISCUSSION OF RESULTS

This section has presented 2 networks, ResNet34 and a small network based on the GRU layer. Each network was implemented using the combinator library, and the code for the combinator-based versions has been presented. The ResNet34 implementation shows a strong correspondence to the graphical description of the network presented. The combinator-based implementation of the GRU highlights the repeated use of learning gateways through the *probe* combinator, and the modularisation of the implementation in the final *GRU* unit, including the forget unit and the network that parameterizes the final gateway and learns what to keep at each timestep. The modularisation also makes visible

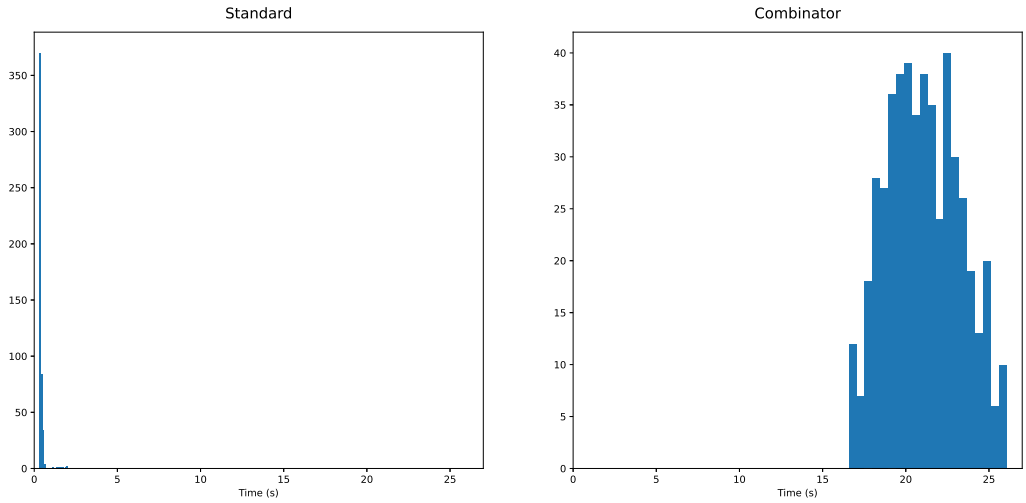


FIGURE 10. A histogram showing a comparison of the time taken to perform epochs of training in the GRU test.

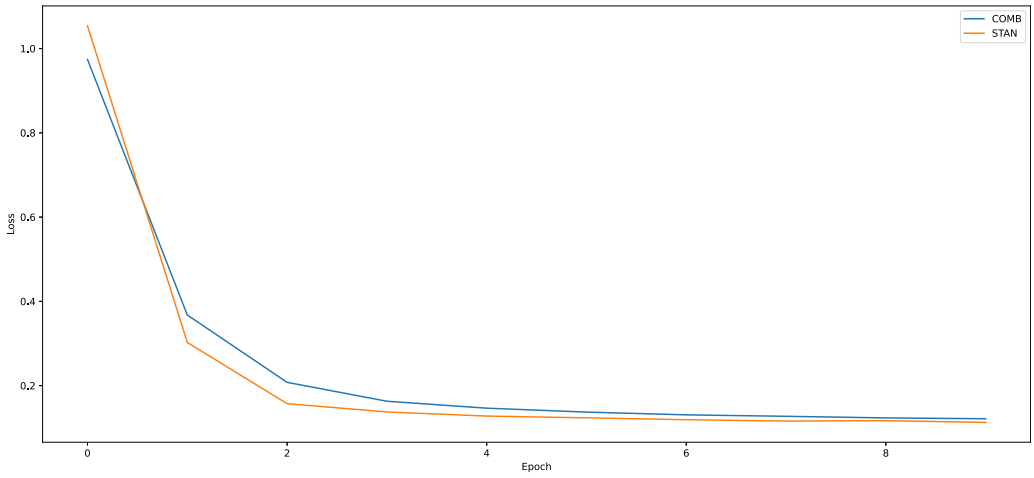


FIGURE 11. A comparison of average training process losses in the GRU implementation.

where networks themselves can be used to parameterize the GRU, in the form of the forget component and the network that parameterizes the final recombination gateway.

ResNet34 has been tested on 2 datasets: the smaller PlantVillage dataset with 12 classes of healthy plant leaves and CIFAR-100 dataset with 100 classes of mixed images. The GRU implementation was tested on the UCI-HAR dataset with 6 classes of human motion.

The combinator-based networks were compared with the standard implementations in terms of the number of trainable parameters, memory usage, training time, training loss. and performance of the trained networks. In all cases, the combinator-based networks had the same number of trainable parameters as the standard networks. When comparing memory allocation, the combinator-based GRU implementation used more memory than the standard implementation, but there was no noticeable difference in the ResNet implementations. This difference in GRU memory usage suggests that nesting components via combinators uses more memory than the standard version.

When trained on the datasets, the combinator implementation of ResNet34 was very close to the standard implementation, though fractionally slower. The training losses found were also very comparable. Comparison of the networks through confusion matrices and a comparison of the predicted classes continued to show strong similarity between the networks' final outputs. The rank and F1-score comparison shown for the CIFAR-100 dataset suggests more differences between the final training of the networks, though even here the F1-scores of each class are similar. The training losses also suggest the networks had not fully converged after 10 epochs, so this demonstrates similar performance and indicates that the combinator implementation behaves like the standard implementation.

The comparison of the training process for the GRU implementations presents greater issues. The combinator



FIGURE 12. A comparison of the average confusion matrix, of the final trained GRU networks, tested on the same evaluation dataset.

implementation was orders of magnitude slower than the standard implementation, suggesting issues in the process used to serialize the batched data and feed the networks during training. The combinator version also did not converge to the same loss level as the standard version, though it was close. The confusion matrices comparing the combinator and standard implementations, when evaluated on the test, suggest similar capability. Where there are classification issues (such as misclassifying standing and sitting), both networks share a blind spot. This is in keeping with the similar losses during training. However, the confusion matrices also confirm that the combinator implementation performs generally worse than the standard example, highlighting an unexpected difference between the implementations.

The introduction noted issues with replicating the training results of any given network, including published ones [14]. This example of testing the GRU illustrates this point. The combinator implementation presented attempted to replicate the structure of an existing version based on the literature and online documentation. However, it was not possible to achieve the same error levels, though it was very close. The slow training times of the combinator solution are a bigger issue. However, we propose that this might be overcome in the future with improved strategies for feeding the datasets to the combinator implementation.

VI. CONCLUSION

In this paper, we have examined various patterns in neural network design and shown how they can be cleanly expressed using functional combinators. We have seen how several common patterns (such as *skip* and *residual*) in the literature are variations on the same underlying pattern and how these common patterns can be abstracted as single combinators, and the common patterns then realized through correct parameterization. The types of these common combinators have been sketched, further highlighting which parameterizations are allowable and how a combinator's parameterization can specialize it. Finally, we have demonstrated the combinator library on the ResNet network design for image classification and on the implementation of the GRU for time series analysis.

In the case of ResNet, the use of combinators does not noticeably affect performance while improving the expression of the design. The GRU example is less clearly an improvement. While it does expose the internal working of the network and shows the repeated use of the *probe* to implement gateways, the performance is very poor when compared to the standard implementations. The tests on the GRU also illustrated the difficulty in exactly replicating the performance of existing designs, even when they are well understood.

In this paper, we have addressed only the implementation of these combinators using PyTorch. PyTorch is the most popular framework, and hence showing that these are useful in PyTorch was sensible. However, a multitude of other frameworks exist that might better suit this functional approach, including the previously mentioned Grenade [5], Jax and Haiku [4], [43], and Flux.jl in the Julia language [44]. The Penzai [23] library for the Jax framework includes a limited set of combinators for neural network construction, but this paper suggests it could go further.

We have shown the types of several combinators and also pointed out how some of these types are shared with similar libraries found in functional programming [17], [38]. The patterns studied and the combinators suggested in this paper are only a subset of the broader field of neural networks; for example, we have not addressed the construction of self-attention or transformers. A key future direction of research will be to draw further inspiration from combinators already used in functional programming and to investigate whether they can be applied to neural network design, while also exploring additional architectures in neural networks and identifying commonalities among them.

Other kinds of networks exist that may not fit the combinator concept at all, because they are based more on training strategy than the pattern combinations applied here. Most obviously, there are 2-stage networks, such as Generative Adversarial and Actor-Critic. Both of these network designs operate by having 2 types of networks performing different functions, and at different times, each is either locked to avoid training or unlocked to allow training. While functions can obviously be written to do this, it is not clear that they fit the style being proposed in this paper and further investigation of these network types should be conducted.

Finally, this work has sought to demonstrate how neural networks can be expressed using combinators and the common patterns that emerge with this approach. However, tests on the current library show efficiency issues. While the ResNet implementation was quite competitive, the GRU version was clearly inferior. While this particular example might be due to the training process itself, there remains a concern that the nesting of modules, as used here, will be less efficient or interact poorly with existing PyTorch optimizations. In terms of memory usage the combinator GRU implementation uses slightly more memory than the standard implementation, but the ResNet34 implementations showed no substantial difference, however the difference seen in the GRU implementation warrants further investigation.

The library's performance has not been investigated in detail but should be further examined in future work. Work in functional compilers such as Haskell with compiler rules have previously shown that functional implementations can combine expressiveness and efficiency [45]. Substantial improvements have also been made recently in PyTorch's performance [46], through algorithms that capture the computation graph dynamically for improved low-level compilation. Python's reflective nature itself can be leveraged to support these activities, by writing combinators that examine their inputs and select appropriate underlying

constructions. For example the *seq* combinator shown in this paper can look at the networks it is asked to combine and efficiently flatten them where appropriate. Another example is using a skip connection with concatenation, followed by a linear layer. This may seem more logical to a programmer, but it is less efficient for the computer. An alternative implementation would be to apply 2 linear layers and sum their outputs. Future work should explore and rigorously characterize these options, considering memory usage and existing PyTorch optimizations in addition to pure throughput performance.

DECLARATION OF COMPETING INTEREST

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

ACKNOWLEDGMENT

The author thank Jörgen Hansen, Amena Darwish, Gunnar Mathiason & Simon Butler for their encouragement and helpful comments on earlier drafts of this work.

DECLARATION OF GENERATIVE AI AND AI-ASSISTED TECHNOLOGIES IN THE MANUSCRIPT PREPARATION PROCESS

During the preparation of this work the author(s) used Anthropic's Claude in order to review sections of this text and make suggestions, but not to generate text directly. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the published article.

REFERENCES

[1] A. Shrestha and A. Mahmood, "Review of deep learning algorithms and architectures," *IEEE Access*, vol. 7, pp. 53040–53065, 2019.

[2] M. Abadi et al. (2015). *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. Software. [Online]. Available: https://www.tensorflow.org/

[3] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Red Hook, NY, USA: Curran Associates Inc., 2019, pp. 8026–8037.

[4] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang. (2018). *JAX: Composable Transformations of Python+NumPy Programs*. [Online]. Available: http://github.com/jax-ml/jax

[5] H. Campbell. (2017). *Grenade: Practical Deep Learning in Haskell*. Hackage Package. [Online]. Available: https://hackage.haskell.org/package/grenade

[6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[7] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention—MICCAI 2015*, N. Navab, J. Hornegger, W. M. Wells, and A. F. Frangi, Eds., Cham, Switzerland: Springer, 2015, pp. 234–241.

[8] K. Chen, *Deep and Modular Neural Networks*. Berlin, Germany: Springer, 2015, pp. 473–494, doi: 10.1007/978-3-662-43505-2_28.

[9] J. Andreas, M. Rohrbach, T. Darrell, and D. Klein, "Neural module networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 39–48.

[10] W. Hu, S. Yang, W. Guo, N. Xiao, X. Yang, and X. Ren, "STC-UNet: Renal tumor segmentation based on enhanced feature extraction at different network levels," *BMC Med. Imag.*, vol. 24, no. 179, Jul. 2024, doi: 10.1186/s12880-024-01359-5.

[11] X.-J. Mao, C. Shen, and Y.-B. Yang, "Image restoration using very deep convolutional encoder–decoder networks with symmetric skip connections," in *Proc. 30th Int. Conf. Neural Inf. Process. Syst.* Red Hook, NY, USA: Curran Associates Inc., 2016, pp. 2810–2818.

[12] J. Yeo. (2016). *Image Restoration Using Convolutional Auto-Encoders With Symmetric Skip Connections: Code*. pyTorch Reimplementation. [Online]. Available: https://github.com/yjn870/REDNet-pytorch

[13] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in *Proc. 32nd AAAI Conf. Artif. Intell. 13th Innov. Appl. Artif. Intell. Conf. 8th AAAI Symp. Educ. Adv. Artif. Intell. AAAI'18/IAAI'18/EAAI*. Washington, DC, USA: AAAI Press, 2018, pp. 3207–3214.

[14] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché Buc, E. Fox, and H. Larochelle, "Improving reproducibility in machine learning research (a report from the neurips 2019 reproducibility program)," *J. Mach. Learn. Res.*, vol. 22, no. 1, pp. 7459–7478, Jan. 2021.

[15] G. Hutton and E. Meijer, "Functional pearl: Monadic parsing in Haskell," *J. Funct. Program. JFP*, vol. 8, no. 4, pp. 437–444, Jul. 1998.

[16] G. Hutton, "A tutorial on the universality and expressiveness of fold," *J. Funct. Program.*, vol. 9, no. 4, pp. 355–372, Jul. 1999.

[17] J. Hughes, "Programming with arrows," in *Advanced Functional Programming*, vol. 3622, V. Vene and T. Uustalu, Eds., Berlin, Germany: Springer, 2005, pp. 73–129.

[18] B. Fong, D. Spivak, and R. Tuyéras, "Backprop as functor: A compositional perspective on supervised learning," in *Proc. 34th Annu. ACM/IEEE Symp. Log. Comput. Sci. (LICS)*, Jun. 2019, pp. 1–13.

[19] C. Elliott, "A functional reboot for deep learning," Invited Talk, BOB Konferenz, Berlin, 2019. [Online]. Available: https://github.com/conal/talk-2018-deep-learning-rebooted

[20] C. Olah. (2015). *Neural Networks, Types, and Functional Programming*. Accessed: Jan. 20, 2026. [Online]. Available: https://colah.github.io/posts/2015-09-NN-Types-FP/

[21] B. Gavranović, "Fundamental components of deep learning: A category-theoretic approach," Ph.D. dissertation, Dept. Comput. Inf. Sci., Univ. Strathclyde, Glasgow, U.K., 2024. [Online]. Available: https://pureportal.strath.ac.uk/en/studentTheses/fundamental-components-of-deep-learning-a-category-theoretic-appr

[22] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 11966–11976, doi: 10.1109/CVPR52688.2022.01167.

[23] D. D. Johnson, "Penzai + Treescope: A toolkit for interpreting, visualizing, and editing models as data," in *Proc. ICML Workshop Mechanistic Interpretability*, 2024.

[24] M. Looks, M. Herreshoff, D. Hutchins, and P. Norvig, "Deep learning with dynamic computation graphs," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017.

[25] S. Stites, H. Zimmermann, H. Wu, E. Sennesh, and J.-W. van de Meent, "Learning proposals for probabilistic programs with inference combinators," in *Proc. 37th Conf. Uncertainty Artif. Intell.*, vol. 161, C. de Campos and M. H. Maathuis, Eds., Jul. 2021, pp. 1056–1066. [Online]. Available: https://proceedings.mlr.press/v161/stites21a.html

[26] J.-P. Bernardy. *TypedFlow: Typed Frontend to Tensorflow and Higher-Order Deep Learning*. Accessed: Aug. 20, 2026. [Online]. Available: https://hackage.haskell.org/package/typedflow

[27] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola, *Dive Into Deep Learning*. Cambridge, U.K.: Cambridge Univ. Press, 2023. [Online]. Available: https://D2L.ai

[28] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 2261–2269.

[29] R. K. Srivastava, K. Greff, and J. Schmidhuber, "Training very deep networks," in *Proc. 29th Int. Conf. Neural Inf. Process. Syst.* Cambridge, MA, USA: MIT Press, 2015, pp. 2377–2385.

[30] A. Liu et al., "Deepseek-v3 technical report," 2025, *arXiv:2412.19437*.

[31] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017. [Online]. Available: https://openreview.net/forum?id=SJU4ayYgl
[32] S. K. Roy, R. Mondal, M. E. Paoletti, J. M. Haut, and A. Plaza, "Morphological convolutional neural networks for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 8689–8702, 2021.
[33] M. Hattori, N. Kobayashi, and R. Sato, "Gradual tensor shape checking," in *Programming Languages and Systems*, T. Wies, Ed., Cham, Switzerland: Springer, 2023, pp. 197–224.
[34] A. Paszke, D. D. Johnson, D. Duvenaud, D. Vytiniotis, A. Radul, M. J. Johnson, J. Ragan-Kelley, and D. Maclaurin, "Getting to the point: Index sets and parallelism-preserving autodiff for pointful array programming," *Proc. ACM Program. Lang.*, vol. 5, pp. 1–29, Aug. 2021, doi: 10.1145/3473593.
[35] F. Rousseau, L. Drumetz, and R. Fablet, "Residual networks as flows of diffeomorphisms," *J. Math. Imag. Vis.*, vol. 62, no. 3, pp. 365–375, Apr. 2020, doi: 10.1007/s10851-019-00890-3.
[36] M. Schönfinkel, "Über die bausteine der mathematischen logik," *Mathematische Annalen*, vol. 92, no. 3, pp. 305–316, Sep. 1924, doi: 10.1007/BF01448013.
[37] H. B. Curry, "Grundlagen der kombinatorischen logik," *Amer. J. Math.*, vol. 52, no. 3, pp. 509–536, Jul. 1930.
[38] R. Paterson, "A new notation for arrows," *ACM SIGPLAN Notices*, vol. 36, no. 10, pp. 229–240, Oct. 2001, doi: 10.1145/507669.507664.
[39] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers Plant Sci.*, vol. 7, p. 1419, Sep. 2016.
[40] A. Krizhevsky, "Learning multiple layers of features from tiny images," Dept. Comput. Sci., Univ. Toronto, Toronto, ON, Canada, Tech. Rep., 2009. [Online]. Available: https://api.semanticscholar.org/CorpusID:18268744
[41] D. Anguita, A. Ghio, L. Oneto, X. Parra, and J. L. Reyes-Ortiz, "A public domain dataset for human activity recognition using smartphones," in *Proc. Eur. Symp. Artif. Neural Netw.*, 2013. [Online]. Available: https://api.semanticscholar.org/CorpusID:6975432
[42] PyTorch Contributors. (2016). *Torchvision/Models/Resnet.Py–ResNet Implementation in TorchVision*. Accessed: Mar. 19, 2026. [Online]. Available: https://github.com/pytorch/vision/blob/main/torchvision/models/resnet.py
[43] T. Hennigan, T. Cai, T. Norman, and I. Babuschkin. (2020). *Haiku: Sonnet for JAX*. [Online]. Available: http://github.com/deepmind/dm-haiku
[44] M. Innes, "Flux: Elegant machine learning with Julia," *J. Open Source Softw.*, vol. 3, no. 25, p. 602, May 2018, doi: 10.21105/joss.00602.
[45] G. Mainland, R. Leshchinskiy, and S. P. Jones, "Exploiting vector instructions with generalized stream fusion," *ACM SIGPLAN Notices*, vol. 48, no. 9, pp. 37–48, Sep. 2013, doi: 10.1145/2544174.2500601.
[46] J. Ansel, "PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation," in *Proc. 29th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.* New York, NY, USA: Association for Computing Machinery, Apr. 2024, pp. 947–9299, doi: 10.1145/3620665.3640366.



RICHARD SENINGTON received the bachelor's and Ph.D. degrees in computer science from the University of Leeds, U.K., in 2009 and 2013, respectively. He is currently a Lecturer of industrial engineering with the School of Engineering Science, University of Skövde, Sweden. His background is in computer science and optimization. He has participated in several research projects in collaboration in Sweden and across Europe. His research interests include artificial intelligence and optimization.